

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4: Implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL3*)
Assessment Tool Weightage: 30%

QUESTIONS: 15

Total Marks:30

Annexure A

CLASS TEST

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

CLASS TEST

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Annexure A

CLASS TEST

QUESTIONS:

Total Marks: 30

Q1. Define architectural patterns in object-oriented systems. List any two common patterns. Explain how the Model-View-Controller (MVC) architecture improves modularity and maintainability.

[L1, L2, L3]

Q2. What is the Data Access Object (DAO) Pattern? Mention two advantages. Explain how DAO separates database-related operations from application logic.

[L2, L3, L4]

Q3. What are object-oriented design guidelines? Identify any two categories of design guidelines. Explain the role of the Liskov Substitution Principle (LSP) in developing reusable software components.

[L2, L3, L4]

Q4. Define database persistence in object-oriented applications. Mention two persistence mechanisms. Explain how an Object-Relational Mapping (ORM) framework supports persistent object management.

[L1, L2, L3]

Q5. What is an Application Layer? State two major responsibilities. Explain how the Application Layer coordinates use cases without directly implementing domain business rules.

[L2, L3, L4]

Q6. Explain the structure of a three-tier architecture consisting of Presentation, Business, and Data layers. Analyze the flow of a customer transaction through these layers in an e-commerce application.

[L2, L4, L5]

Q7. What is the role of a Business Logic Layer? List two important functions. Explain how it validates and processes requests received from the presentation layer.

[L2, L3, L4]

Q8. Explain the significance of cohesion and coupling in object-oriented design. Analyze how high cohesion and low coupling contribute to software maintainability and reusability.

[L3, L4, L5]

Q9.What is encapsulation in an object-oriented system? Explain how encapsulation protects object state and compare it with data hiding using a suitable example.

[L2, L4]

Q10.Design a layered object-oriented system for an online banking application. Identify the major layers, recommend suitable design patterns, and justify how the architecture improves security, scalability, and maintainability.

[L3, L4, L5]

Q11.What is the role of the Infrastructure Layer in object-oriented architecture? List two responsibilities. Explain how it supports technical services such as database access and external system communication.

[L1, L2, L3]

Q12.Define the Singleton Design Pattern. Mention its major characteristics. Explain its appropriate use in managing shared resources and discuss one limitation of the pattern.

[L2, L3, L4]

Q13.What is dependency injection in object-oriented design? List two benefits. Analyze how dependency injection reduces tight coupling between classes and improves testability.

[L2, L3, L4]

Q14.Explain the working of the Strategy Design Pattern. Compare the roles of Context and Strategy with a suitable real-world application example.

[L3, L4, L5]

Q15.Design a layered architecture for a Library Management System. Identify suitable layers, recommend relevant design patterns, and justify how your proposed architecture supports flexibility and maintainability.

[L3, L4, L5]

Q1. Architectural Patterns and MVC

An architectural pattern is a reusable high-level solution for organizing the structure and interaction of components in an object-oriented software system. It defines how major system components are divided and how they communicate with each other.

Two common architectural patterns are:

1. Model-View-Controller (MVC)
2. Three-Tier Architecture

Model-View-Controller

MVC divides an application into three main components:

- Model: Represents data and business/domain information.
- View: Presents information to the user.
- Controller: Receives user requests and coordinates the Model and View.

For example, in an online shopping application, the product information belongs to the Model, the product page is the View, and the Controller processes the user's request.

MVC improves modularity because each component has a separate responsibility. A change in the user interface can be made in the View without changing the Model. Similarly, business logic can be modified independently of the presentation.

It also improves maintainability because the code is organized into clearly defined components. Testing becomes easier because individual components can be tested separately. MVC also supports reuse because the same Model can be used with different Views.

Thus, architectural patterns provide a structured approach to developing flexible, maintainable, and reusable object-oriented applications.

Q2. Data Access Object (DAO) Pattern

The Data Access Object (DAO) Pattern is a design pattern that separates database access operations from the application's business logic. A DAO provides methods for creating, reading, updating, and deleting data without exposing database implementation details to other classes.

For example, a StudentDAO class may contain methods such as:

- addStudent()
- getStudent()
- updateStudent()
- deleteStudent()

Advantages

1. Separation of concerns: Database operations are separated from business logic.
2. Easy maintenance: Database implementation can be changed without modifying the application logic.

The application service communicates with the DAO instead of directly executing database queries. The DAO handles database connections, queries, transactions, and conversion of database records into objects.

For example:

Application Layer

|

v

StudentDAO

|

v

Database

If the application changes from one database technology to another, only the DAO implementation may need modification.

Therefore, DAO reduces coupling between the application and database and provides a clean and maintainable architecture.

Q3. Object-Oriented Design Guidelines and LSP

Object-oriented design guidelines are principles and recommended practices used to create software that is reusable, maintainable, flexible, understandable, and easy to test.

Two important categories are:

1. Class and object design guidelines
2. Relationship and inheritance design guidelines

Class and object guidelines focus on responsibilities, encapsulation, cohesion, and appropriate class design. Relationship and inheritance guidelines focus on inheritance, abstraction, coupling, and substitutability.

Liskov Substitution Principle

The Liskov Substitution Principle (LSP) states that objects of a subclass should be replaceable for objects of their superclass without changing the correctness of the program.

For example, consider:

Vehicle

|

+---- Car

|

+---- Bike

If a program expects a Vehicle, both Car and Bike should behave consistently with the expected behavior of a Vehicle.

A common violation occurs when a subclass inherits an operation but cannot meaningfully support it. This indicates that the inheritance relationship is incorrectly designed.

LSP helps developers create reusable components because subclasses can safely be used wherever the parent class is expected. It also reduces unexpected behavior and improves flexibility.

Thus, following LSP results in better inheritance structures, reusable classes, and maintainable object-oriented software.

Q4. Database Persistence and ORM

Database persistence is the ability of an object's data to remain available even after the application that created the object has stopped running. Persistent objects are stored in permanent storage such as a database.

Two common persistence mechanisms are:

1. Object-Relational Mapping (ORM)
2. Direct database access using SQL and data-access APIs

ORM

An Object-Relational Mapping framework provides a connection between object-oriented classes and relational database tables. It maps classes to tables, objects to rows, and object attributes to columns.

For example:

Student Object

|

v

Student Class

|

v

Student Table

An ORM framework can automatically perform operations such as inserting, retrieving, updating, and deleting objects.

For example, a Student object can be stored as a record in a Student table without requiring the programmer to manually write SQL for every operation.

ORM supports persistent object management by maintaining object identity, handling database transactions, converting database records into objects, and storing object changes.

Therefore, ORM reduces database-related code and allows developers to work with persistent data using object-oriented concepts.

Q5. Application Layer

The Application Layer is the layer responsible for coordinating application-specific use cases and controlling the flow of operations between the user interface and domain objects.

Two major responsibilities are:

1. Coordinating use cases
2. Managing application-level operations and transactions

The Application Layer receives a request from the Presentation Layer and determines the sequence of operations required to complete the use case.

For example, in an online banking system, when a customer requests a fund transfer, the Application Layer coordinates the transfer process by calling appropriate domain services and repositories.

Presentation Layer

|

v

Application Layer

|

v

Domain Layer

|

v

Infrastructure Layer

The Application Layer should not contain detailed business rules such as calculating interest or determining loan eligibility. Those rules belong to the Domain or Business Layer.

Instead, it coordinates the required domain operations.

This separation improves maintainability because changes to application workflows can be made without mixing them with business rules or user-interface code.

Q6. Three-Tier Architecture in an E-Commerce Application

A three-tier architecture divides an application into three major layers:

1. Presentation Layer
2. Business Layer
3. Data Layer

1. Presentation Layer

This is the user interface of the application. It allows customers to browse products, add products to a cart, enter payment information, and view order status.

2. Business Layer

This layer contains business rules and application processing. It validates orders, calculates prices, applies discounts, checks inventory, and processes transactions.

3. Data Layer

This layer communicates with the database and manages persistent information such as customers, products, orders, and payments.

Transaction Flow

Consider a customer purchasing a product:

Customer

|

v

Presentation Layer

|

v

Business Layer

|

v

Data Layer

|

v

Database

First, the customer selects a product and submits an order through the Presentation Layer. The request is passed to the Business Layer.

The Business Layer validates the customer details, checks product availability, calculates the order amount, applies applicable discounts, and processes the order.

The Business Layer then requests the Data Layer to store the order and update the product stock.

The Data Layer performs the required database operations.

The result is returned through the Business Layer to the Presentation Layer, where the customer receives an order confirmation.

This architecture improves scalability because each layer can be developed and maintained independently. It also improves security because direct database access is restricted to the Data Layer.

Q7. Business Logic Layer

The Business Logic Layer contains the rules and processing required to implement the actual business requirements of an application.

Two important functions are:

1. Validating business rules
2. Processing business operations

For example, in an online shopping system, the Business Logic Layer verifies whether a product is available, calculates the total price, applies discounts, and validates payment conditions.

The processing flow is:

Presentation Layer

|

v

Business Logic Layer

|

v

Data Access Layer

|

v

Database

When a request is received from the Presentation Layer, the Business Logic Layer validates the request. It checks whether the input satisfies the required business rules.

If the request is valid, it performs the required operation and communicates with the Data Access Layer when data needs to be retrieved or stored.

For example, during a purchase, it checks stock availability before confirming the order.

Separating business logic from presentation code makes the application easier to maintain, test, and reuse.

Q8. Cohesion and Coupling

Cohesion refers to how closely related the responsibilities within a class or module are. Coupling refers to the level of dependency between different classes or modules.

A good object-oriented design aims for:

- High cohesion
- Low coupling

High Cohesion

A class has high cohesion when its methods and attributes are strongly related to one clear responsibility.

For example, a `PaymentService` should primarily handle payment-related operations rather than customer registration and inventory management.

High cohesion makes classes easier to understand, test, modify, and reuse.

Low Coupling

Low coupling means that classes have minimal dependency on the internal implementation of other classes.

For example:

`OrderService` ---> `PaymentService`

The `OrderService` should depend only on the required payment interface rather than depending on the internal implementation of `PaymentService`.

Importance

High cohesion improves maintainability because changes are generally restricted to the class responsible for that functionality.

Low coupling improves reusability because a class can be used in different parts of the application without requiring many dependent classes.

Together, high cohesion and low coupling produce flexible, maintainable, reusable, and testable software.

Q9. Encapsulation and Data Hiding

Encapsulation is an object-oriented principle that combines data and the methods operating on that data within a single class. It controls how the object's internal state can be accessed or modified.

For example:

```
class BankAccount
```

```
    private balance
```

```
    deposit()
```

```
    withdraw()
```

```
    getBalance()
```

The balance should not be directly modified by other classes. Instead, the deposit() and withdraw() methods control changes to the balance.

This protects the object from invalid states. For example, the withdraw() method can prevent withdrawal when sufficient balance is not available.

Encapsulation vs Data Hiding

Encapsulation focuses on combining data and related operations into a single unit and controlling access through methods.

Data hiding focuses specifically on preventing direct access to the internal implementation or data of an object.

For example:

```
BankAccount
```

```
|
```

```
+-- private balance
```

```
|
```

```
+-- deposit()
```

```
|
```

```
+-- withdraw()
```

Here, encapsulation combines the balance and related operations within the BankAccount class, while data hiding prevents other classes from directly accessing the private balance.

Therefore, both concepts improve security, maintainability, and reliability of object-oriented systems.

Q10. Layered Object-Oriented System for Online Banking

An online banking system should use a layered architecture to separate presentation, application processing, business rules, and technical services.

Proposed Layers

Presentation Layer

|

Application Layer

|

Business / Domain Layer

|

Data Access Layer

|

Infrastructure Layer

|

Database / External Services

1. Presentation Layer

Provides login, account details, balance enquiry, fund transfer, and transaction interfaces.

2. Application Layer

Coordinates use cases such as fund transfer, bill payment, account management, and transaction history.

3. Business/Domain Layer

Contains important banking rules such as account validation, transaction limits, balance checking, and transfer rules.

4. Data Access Layer

Uses DAO or Repository patterns to access customer, account, and transaction data.

5. Infrastructure Layer

Provides database connectivity, authentication services, logging, messaging, and communication with external payment systems.

Suitable Design Patterns

- MVC Pattern – separates user interface, control logic, and data representation.
- DAO/Repository Pattern – separates database operations from business logic.
- Factory Pattern – creates appropriate service objects.
- Strategy Pattern – supports different payment or transaction strategies.
- Dependency Injection – reduces coupling between components.

Security

Layered separation prevents the Presentation Layer from directly accessing sensitive database information. Authentication and authorization can be implemented in appropriate service layers.

Scalability

Individual layers can be independently modified or scaled. For example, additional application servers can handle increased transaction requests.

Maintainability

Each layer has a clear responsibility. Changes to the database, user interface, or business rules can be made with minimal impact on other layers.

Thus, layered architecture provides a secure, scalable, flexible, and maintainable foundation for online banking.

Q11. Infrastructure Layer

The Infrastructure Layer provides technical services required by the application but does not contain the core business rules.

Two major responsibilities are:

1. Database and persistence services
2. External system communication

The Infrastructure Layer implements technical interfaces required by higher-level application and domain layers.

For database access, it provides database connections, repositories, DAO implementations, transaction management, and persistence operations.

For external communication, it can communicate with payment gateways, email services, authentication providers, messaging systems, and third-party APIs.

For example:

Business Layer

|

v

Repository Interface

|

v

Infrastructure Implementation

|

+----> Database

|

+----> External API

This separation prevents business classes from depending directly on database technologies or external APIs.

The Infrastructure Layer therefore supports technical operations while allowing the business and application layers to remain independent of specific technologies.

Q12. Singleton Design Pattern

The Singleton Design Pattern ensures that a class has only one instance throughout the application and provides a controlled way to access that instance.

Major Characteristics

1. Only one instance of the class is created.
2. The instance is accessed through a common access point.
3. The constructor is generally restricted so that other classes cannot freely create instances.

A Singleton can be useful for managing shared resources such as configuration management, application-wide logging, or a carefully controlled shared service.

For example:

Application

|

v

Configuration Manager

|

+---- Single Instance

Using a single configuration manager ensures that the application accesses a consistent configuration object.

Limitation

A major limitation is that Singleton introduces global state and can create hidden dependencies between classes. It can also make unit testing more difficult because classes may directly depend on the single instance.

Therefore, Singleton should be used only when a single shared instance is genuinely required.

Q13. Dependency Injection

Dependency Injection (DI) is a design technique in which an object's required dependencies are provided from outside instead of the object creating those dependencies itself.

For example, instead of:

OrderService creates PaymentService

the dependency is provided externally:

OrderService <--- PaymentService

Two major benefits are:

1. Reduced coupling
2. Improved testability

Without dependency injection, a class may directly create a specific implementation, making it difficult to replace.

With DI, the class can depend on an interface:

PaymentService

^

|

PaymentInterface

^

|

OrderService

A real payment implementation can be provided in production, while a mock implementation can be provided during testing.

This reduces tight coupling because the dependent class does not control the creation of its dependency.

It also improves testability because fake or mock objects can easily be injected during unit testing.

Therefore, dependency injection supports flexible, reusable, maintainable, and testable object-oriented software.

Q14. Strategy Design Pattern

The Strategy Design Pattern defines a family of algorithms, places each algorithm in a separate class, and allows the algorithm to be selected or changed at runtime.

The main participants are:

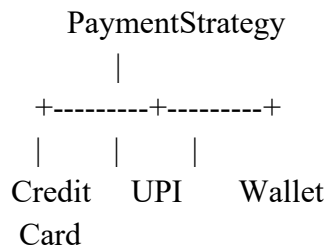
Context

The Context is the class that uses a strategy. It does not need to know the internal implementation of the algorithm.

Strategy

The Strategy defines a common interface for different algorithms.

For example, consider an online shopping application that supports different payment methods:



The Context may be PaymentProcessor.

The Strategy interface defines:

processPayment()

Different strategy classes implement this operation according to their payment method.

The customer can select a payment method, and the Context uses the corresponding Strategy.

Working

1. The application creates the Context.
2. A suitable Strategy is selected.
3. The Strategy is assigned to the Context.
4. The Context calls the common Strategy operation.
5. The selected Strategy performs the actual operation.

The major advantage is that new strategies can be added without significantly changing the Context.

Thus, Strategy supports flexibility, reduces conditional logic, and follows the principle of programming to an abstraction.

Q15. Layered Architecture for Library Management System

A Library Management System can be designed using a layered object-oriented architecture to manage students or members, books, borrowing, returns, reservations, and fines.

Proposed Architecture

Presentation Layer

Application Layer

Domain / Business Layer

Data Access Layer

|
Infrastructure Layer

|
Database

1. Presentation Layer

Provides interfaces for:

- Member login
- Book search
- Book reservation
- Book borrowing
- Book return
- Fine information
- Librarian operations

2. Application Layer

Coordinates use cases such as:

- Register member
- Search book
- Issue book
- Return book
- Reserve book
- Calculate and process fines

It coordinates operations without directly implementing the detailed business rules.

3. Domain/Business Layer

Contains classes such as:

- Book
- Member
- Librarian
- Loan
- Reservation
- Fine

It contains rules such as maximum borrowing limits, book availability, return deadlines, and fine calculation.

4. Data Access Layer

Provides database operations using DAO or Repository Pattern.

For example:

BookRepository

MemberRepository

LoanRepository

ReservationRepository

5. Infrastructure Layer

Handles database connections, persistence, logging, authentication, notifications, and external services.

Suitable Design Patterns

MVC Pattern: Separates user interface, control operations, and application data.

DAO/Repository Pattern: Separates database operations from business logic.

Strategy Pattern: Can be used for different fine-calculation strategies or notification methods.

Factory Pattern: Can be used to create different types of users, notifications, or library resources.

Dependency Injection: Provides required services to application classes and reduces tight coupling.

Flexibility

The layered design allows one layer to change without requiring major changes in other layers. For example, the database technology can be changed without modifying the Presentation Layer.

Maintainability

Each layer has a clearly defined responsibility. Business rules remain separate from database and user-interface code, making testing and maintenance easier.

Scalability

New features such as online payment of fines, mobile access, email notifications, and digital-book management can be added without redesigning the entire system.

Conclusion

The proposed layered architecture provides a flexible and maintainable design for the Library Management System. The combination of layered architecture, MVC, Repository/DAO, Strategy, Factory, and Dependency Injection principles provides clear separation of concerns, reduced coupling, improved reusability, and easier future enhancement.